

1% FRESHER

Production-Level Developer Roadmap

GOAL

Build the skillset to enter a job as a fresher with production-level engineering thinking.

What makes a 1% fresher?

Not knowing 50 technologies. The real advantage is being able to design, implement, test, debug, optimize, secure and deploy a complex feature independently.

Mindset: Don't just ask "How do I code this?" Ask "How should this be designed, scaled, tested, secured and maintained?"

■ 3. TypeScript

■ Generics	■ Union / Intersection
■ Type narrowing	■ Type guards
■ Utility types	■ Conditional types
■ keyof	■ typeof
■ Discriminated unions	■ Generic components
■ API typing	■ Zod + TypeScript

■ 4. Software Architecture

■ Feature-based architecture	■ Separation of concerns
■ Component architecture	■ Service layer
■ Reusable components	■ SOLID
■ DRY	■ KISS
■ Composition	■ Abstraction
■ Dependency management	■ Refactoring
■ Technical debt	

■ 5. API & HTTP

■ HTTP deeply	■ REST
■ Status codes	■ Headers
■ Cookies	■ CORS
■ Authentication	■ Authorization

■ Pagination	■ Filtering
■ Sorting	■ Retry
■ AbortController	■ Race conditions
■ Debouncing	■ Throttling
■ Rate limiting	

■ 6. State Architecture

■ UI State	■ Client State
■ Server State	■ URL State
■ Form State	■ Zustand
■ Redux Toolkit	■ TanStack Query
■ Context	■ React Hook Form

Rule: Don't put every state into Redux or Context. Choose the right state owner.

■ 7. Performance Engineering

■ Core Web Vitals	■ LCP
■ INP	■ CLS
■ Bundle analysis	■ Code splitting
■ Lazy loading	■ Dynamic imports
■ Image optimization	■ Font optimization
■ Network waterfall	■ React Profiler
■ Chrome Performance	■ Memory leaks
■ Long tasks	

Goal: Diagnose “the app is slow” using evidence from DevTools—not guesses.

■ 8. Security

■ XSS	■ CSRF
■ CORS	■ CSP
■ Same-Origin Policy	■ HTTP-only cookies
■ Secure cookies	■ SameSite
■ Authentication	■ Authorization
■ RBAC	■ Input validation
■ Rate limiting	■ Secrets management

■■ 9. Database Fundamentals

■ PostgreSQL

■ Relations

■ Joins

■ Indexes

■ Transactions

■ Constraints

■ Pagination

■ Query optimization

■ N+1 problem

■ Prisma / Drizzle

■ 10. Testing

■ Unit testing

■ Integration testing

■ E2E testing

■ Vitest / Jest

■ React Testing Library

■ MSW

■ Playwright

■ 11. Debugging

■ Chrome DevTools

■ Network tab

■ Performance tab

■ Memory

■ React DevTools

■ Breakpoints

■ Call stack

■ API debugging

■ Production debugging

Goal: *Reproduce → inspect → identify root cause → fix → test → prevent regression.*

■■■ 12. Git & Code Review

■ Branching

■ Rebase

■ Cherry-pick

■ Revert

■ Bisect

■ Conflict resolution

■ Pull Requests

■ Code reviews

■ Conventional commits

■ 13. System Design — Fresher Selected

■ Client → Server architecture

■ CDN

■ Load Balancer

■ Caching

■ Redis basics

■ Database scaling concepts

■ Horizontal vs Vertical scaling

■ Stateless servers

■ Queues basics

■ WebSockets

■ Rate limiting

■ Database indexing

■ Read/write separation concepts

Target: Understand the architecture well enough to explain why each layer exists and what trade-off it creates.

■ 14. Production Engineering

■ Environment variables

■ CI/CD

■ Docker basics

■ Vercel

■ GitHub Actions

■ Error monitoring

■ Logging

■ Sentry

■ Production builds

■ Rollbacks

■ Deployment strategy

■ 15. AI Application Engineering

■ Streaming responses

■ SSE

■ WebSockets

■ Chat architecture

■ Token streaming

■ Abort generation

■ Retry

■ Rate-limit handling

■ Conversation persistence

■ File uploads

■ Markdown rendering

■ Optimistic UI

■ AI error handling

■ TOP 10 — If Time Is Limited

Rank	Skill
■	React internals + debugging
■	Next.js architecture
■	TypeScript
4	Software Architecture
5	API / HTTP
6	Performance
7	State Architecture
8	Security
9	Testing
10	Git + CI/CD

■ Final Rule

A 1% fresher is not the person who memorizes the most APIs. It is the person who can take a complex feature from **design** → **implementation** → **testing** → **optimization** → **security** → **deployment** → **debugging**.

Build fewer, larger projects. Every project should force you to use architecture, TypeScript, APIs, state management, performance, security, testing, Git and deployment.